

# NEURAL NETWORKS

## Foundations: Neural Networks to Representation Learning

Author: Eng. Carlos Andrés Sierra, M.Sc.  
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor  
Computer Engineering Program  
School of Engineering  
Universidad Distrital Francisco José de Caldas

2026-I



UNIVERSIDAD DISTRITAL  
FRANCISCO JOSÉ DE CALDAS

# Outline

1 Neural Network Fundamentals

2 Optimization & Regularization



# Outline

1 Neural Network Fundamentals

2 Optimization & Regularization

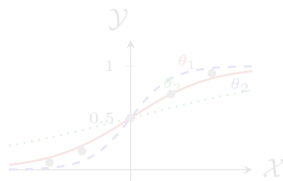


# What is a Neural Network?

## Definition

A neural network is a parametric function  $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$  composed of simple, interconnected computational units (neurons).

- **Neuron:** weighted sum + nonlinear activation
- **Layer:** neurons operating in parallel
- **Depth:** key enabler of hierarchical features
- **Training:** adjust  $\theta$  so  $f_{\theta}(x) \approx y$



Different  $\theta \Rightarrow$  different  $f_{\theta}$ .

Training finds the best fit.



# What is a Neural Network?

⊖ ~~Not~~ convex

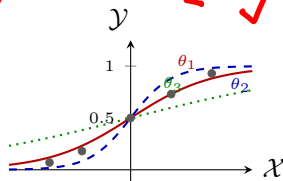
## Definition

A neural network is a **parametric function**  $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$  composed of simple, interconnected computational units (*neurons*).

- **Neuron:** weighted sum + nonlinear activation
- **Layer:** neurons operating in parallel

- **Depth:** layers of hierarchical features
- **Training:** adjust  $\theta$  so  $f_{\theta}(x) \approx y$

⊖ ~~Not~~ convex  
 $w_1 + w_2 = 1$



Different  $\theta \Rightarrow$  different  $f_{\theta}$ .

Training finds the best fit.

$$f(x_1, x_2) = w_1 \cdot x_1 + w_2 \cdot x_2$$



# What is a Neural Network?

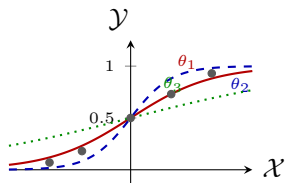
## Definition

A neural network is a **parametric function**  $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$  composed of simple, interconnected computational units (*neurons*).

- **Neuron:** weighted sum + nonlinear activation
- **Layer:** neurons operating in parallel

- **Depth:** key enabler of hierarchical features
- **Training:** adjust  $\theta$  so  $f_{\theta}(x) \approx y$

Parallel



Different  $\theta \Rightarrow$  different  $f_{\theta}$ .

Training finds the best fit.

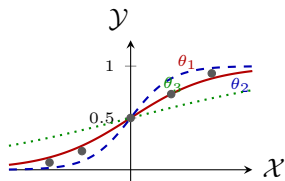


# What is a Neural Network?

## Definition

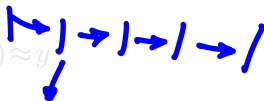
A neural network is a **parametric function**  $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$  composed of simple, interconnected computational units (*neurons*).

- **Neuron:** weighted sum + nonlinear activation
- **Layer:** neurons operating in parallel
- **Depth:** key enabler of hierarchical features
- **Training:** adjust  $\theta$  so  $f_{\theta}(x) \approx y$



Different  $\theta \Rightarrow$  different  $f_{\theta}$ .

Training finds the best fit.

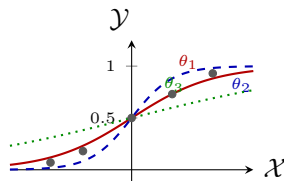


# What is a Neural Network?

## Definition

A neural network is a **parametric function**  $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$  composed of simple, interconnected computational units (*neurons*).

- **Neuron:** **weighted sum** + nonlinear activation
- **Layer:** neurons operating in parallel
- **Depth:** key enabler of **hierarchical features**
- **Training:** adjust  $\theta$  so  $f_{\theta}(x) \approx y$



Different  $\theta \Rightarrow$  different  $f_{\theta}$ .

Training finds the best fit.

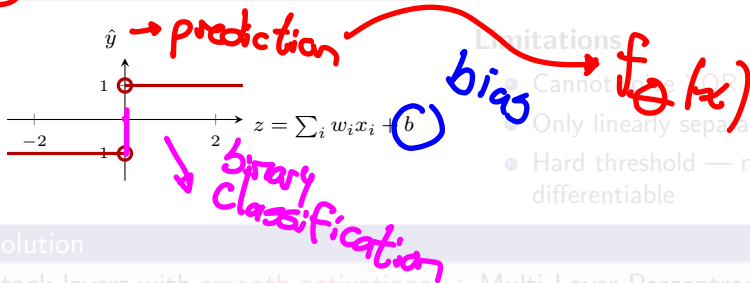
$w_1, w_2 \rightarrow 47$   
 $\theta_3 \rightarrow b_i$   
 Points



# The Perceptron

Rosenblatt (1958)

$$\hat{y} = \text{sign}(\sum_i w_i x_i + b)$$



Strengths

- Binary classification
- Convergence guaranteed (linearly separable data)

Limitations

- Cannot solve XOR
- Only linearly separable
- Hard threshold — not differentiable

Solution

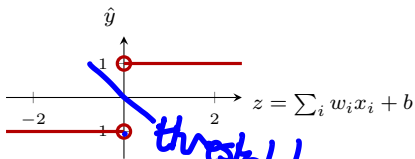
Stack layers with smooth activations  $\Rightarrow$  Multi-Layer Perceptron



# The Perceptron

Rosenblatt (1958)

$$\hat{y} = \text{sign}(\sum_i w_i x_i + b)$$



AND-OR

Solution

Stack layers with smooth activations  $\Rightarrow$  Multi-Layer Perceptron

## Strengths

- Binary classification ✓
- Convergence guarantee (linearly separable data) ✓

## Limitations

- Cannot solve XOR
- Only linearly separable
- Hard threshold — not differentiable

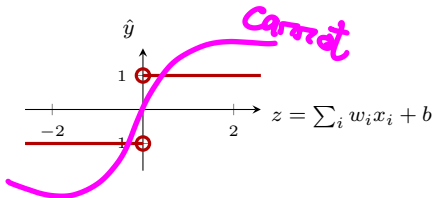
Linear



# The Perceptron

Rosenblatt (1958)

$$\hat{y} = \text{sign}(\sum_i w_i x_i + b)$$



## Strengths

- Binary classification
- Convergence guarantee (linearly separable data)

## Limitations

- Cannot solve XOR
- Only linearly separable
- Hard threshold — not differentiable

## Solution

Stack layers with smooth activations  $\Rightarrow$  Multi-Layer Perceptron ✓



# Multi-Layer Perceptron (MLP) — Architecture

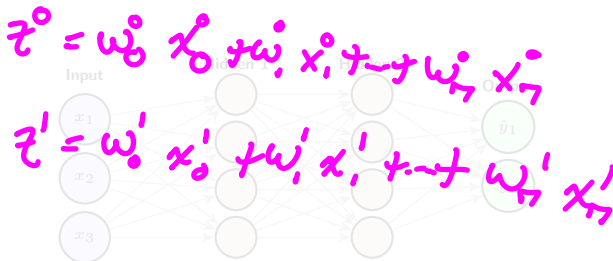
## Architecture

Input layer → Hidden layer(s) → Output layer

For each layer  $l$ :

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$

at least  
3 layers



• Universal Approximation (Hornik, 1991): one hidden layer with enough neurons approximates any continuous function  
 • Depth is more parameter-efficient than extreme width



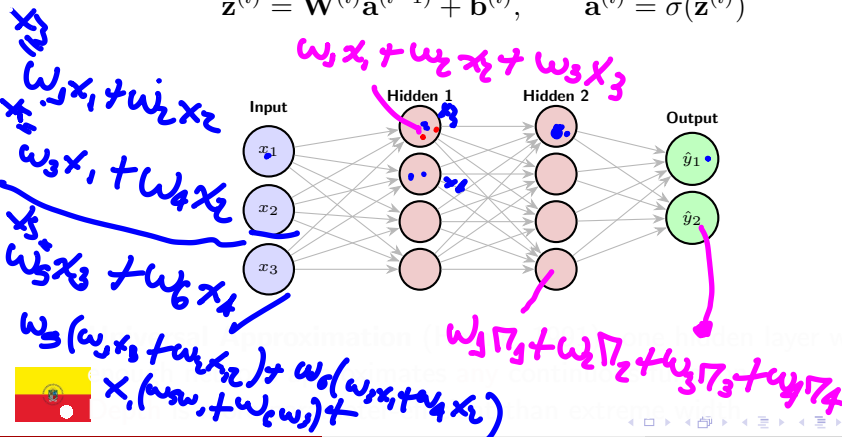
# Multi-Layer Perceptron (MLP) — Architecture

## Architecture

Input layer → Hidden layer(s) → Output layer

For each layer  $l$ :

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$



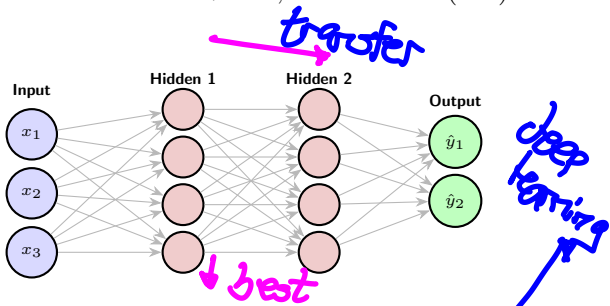
# Multi-Layer Perceptron (MLP) — Architecture

## Architecture

Input layer → Hidden layer(s) → Output layer

For each layer  $l$ :

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$



**Universal Approximation** (Hornik, 1991): one hidden layer with enough neurons approximates any continuous function

**Depth** is more parameter-efficient than extreme width



# Multi-Layer Perceptron (MLP) — Output & Softmax

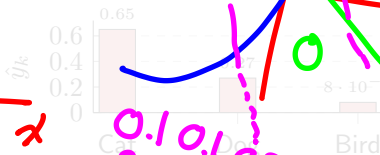
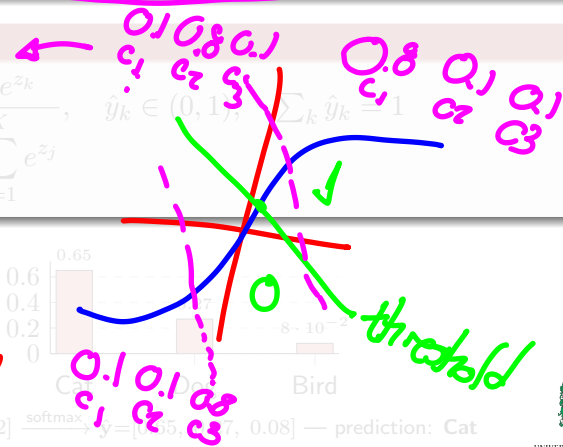
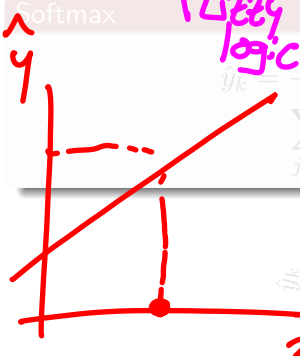
## Output layer choices

- **Linear:** regression
- **Sigmoid:** binary classification
- **Softmax:** multi-class → probability distribution over  $K$  classes

## Softmax

Fuzzy logic

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad \hat{y}_k \in (0, 1), \quad \sum_k \hat{y}_k = 1$$



$z = [2.1, 0.3, -1.2] \xrightarrow{\text{softmax}} y = [0.65, 0.27, 0.08] \text{ — prediction: Cat}$



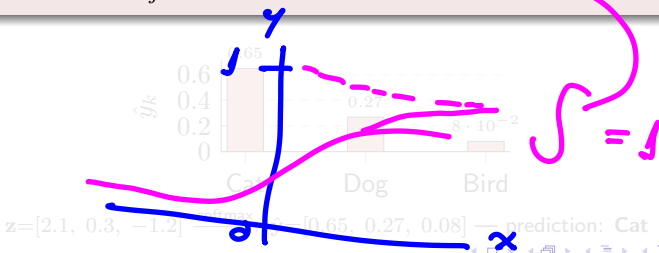
# Multi-Layer Perceptron (MLP) — Output & Softmax

## Output layer choices

- **Linear**: regression     **Sigmoid**: binary classification
- **Softmax**: multi-class  $\rightarrow$  probability distribution over  $K$  classes

## Softmax

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad \hat{y}_k \in (0, 1), \quad \sum_k \hat{y}_k = 1$$



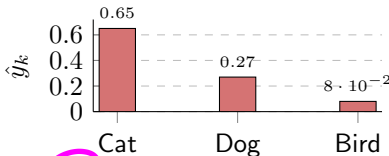
# Multi-Layer Perceptron (MLP) — Output & Softmax

## Output layer choices

- **Linear**: regression     **Sigmoid**: binary classification
- **Softmax**: multi-class  $\rightarrow$  probability distribution over  $K$  classes

## Softmax

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad \hat{y}_k \in (0, 1), \quad \sum_k \hat{y}_k = 1$$



*Fuzzy logic*

$\mathbf{z} = [2.1, 0.3, -1.2] \xrightarrow{\text{softmax}} \hat{\mathbf{y}} = [0.65, 0.27, 0.08] \text{ — prediction: Cat}$



# Activation Functions

**Sigmoid:**

$$\sigma(x) = \frac{1}{1 + e^{-x}} \in (0, 1)$$

**Tanh:**

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \in (-1, 1)$$

**ReLU:**  $\max(0, x)$  — **default today**

**GELU:**  $x \Phi(x)$  — used in Transformers

→ factor → neuron output

Why ReLU dominates

- No saturation for positive inputs
- Sparse activation (many zeros)
- Computationally trivial
- Mitigates vanishing gradient

$(w_1 x_1 + w_2 x_2) \cdot \text{af}$



# Activation Functions

**Sigmoid:**

$$\sigma(x) = \frac{1}{1 + e^{-x}} \in (0, 1)$$

**Tanh:**

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \in (-1, 1)$$

**ReLU:**  $\max(0, x)$  — **default today**

**GELU:**  $x \cdot \Phi(x)$  — used in Transformers

## Why ReLU dominates

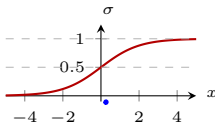
- No saturation for positive inputs
- Sparse activation (many zeros)
- Computationally trivial
- Mitigates vanishing gradient



# Activation Functions — Visual Overview

## Sigmoid

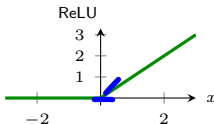
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Output layer

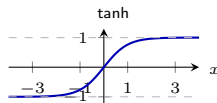
## ReLU

$$\max(0, x)$$



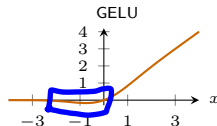
## Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



## GELU

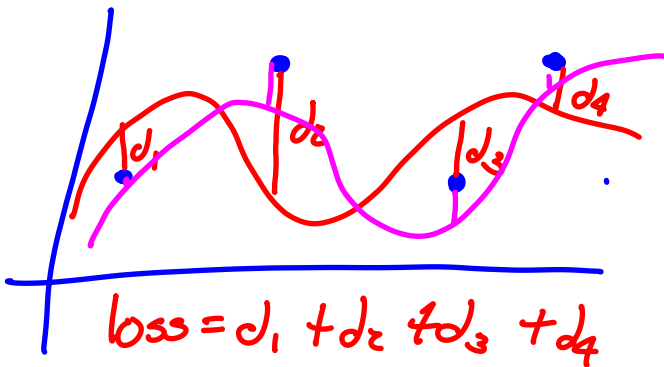
$$x \cdot \Phi(x)$$



# Loss Functions

## The Objective of Training

Minimise  $\mathcal{L}(\theta)$  — a measure of disagreement between predictions and targets over the training set.



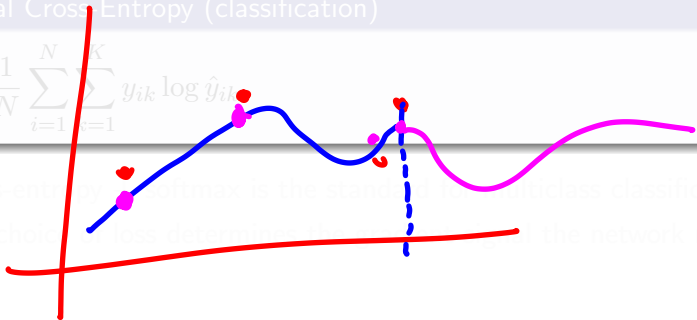
# Loss Functions

## Mean Squared Error (regression)

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\hat{y}_i - y_i\|^2$$

## Categorical Cross Entropy (classification)

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \hat{y}_{ik}$$



- Cross-entropy with softmax is the standard loss for multiclass classification
- The choice of loss determines the metric that the network receives



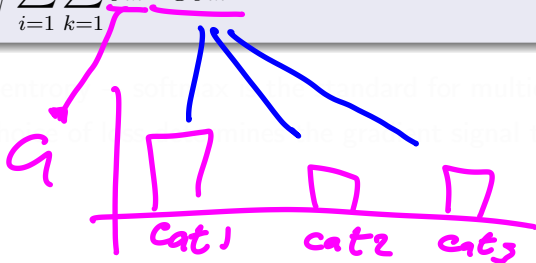
# Loss Functions

## Mean Squared Error (regression)

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\hat{y}_i - y_i\|^2$$

## Categorical Cross-Entropy (classification)

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \hat{y}_{ik}$$



# Loss Functions

## Mean Squared Error (regression)

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\hat{y}_i - y_i\|^2$$

## Categorical Cross-Entropy (classification)

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \hat{y}_{ik}$$

- Cross-entropy + softmax is the standard for multiclass classification
- The choice of loss determines the gradient signal the network receives



# Backpropagation

## Core Idea

Apply the chain rule repeatedly to compute  $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$  for every layer, from output back to input.

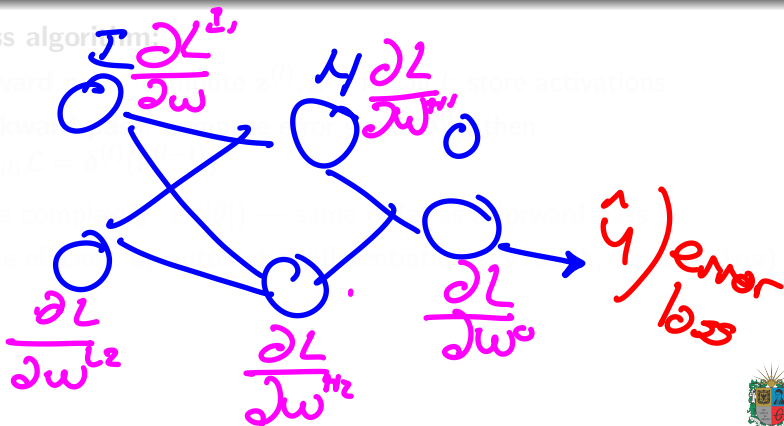
Two-pass algorithm:

• Forward pass: compute all  $\mathbf{z}^{(l)}$  and store activations

• Backward pass: compute all  $\delta^{(l)}$  when

Time complexity is  $O(N^2)$  for fully connected layers

Made



# Backpropagation

## Core Idea

Apply the **chain rule** repeatedly to compute  $\partial \mathcal{L} / \partial \mathbf{W}^{(l)}$  for every layer, from output back to input.

## Two-pass algorithm:

① **Forward pass:** compute  $\mathbf{z}^{(l)}$ ,  $\mathbf{a}^{(l)}$  for all  $l$ ; store activations

② **Backward pass:** compute error signal  $\delta^{(l)}$ , then  

$$\nabla_{\mathbf{W}^{(l)}} \mathcal{L} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$$

• Time complexity:  $O(|\theta|)$  — same order as a forward pass

• Made efficient by automatic differentiation (PyTorch, TensorFlow)



# Backpropagation

## Core Idea

Apply the **chain rule** repeatedly to compute  $\partial \mathcal{L} / \partial \mathbf{W}^{(l)}$  for every layer, from output back to input.

## Two-pass algorithm:

① **Forward pass:** compute  $\mathbf{z}^{(l)}, \mathbf{a}^{(l)}$  for all  $l$ ; store activations

② **Backward pass:** compute error signal  $\delta^{(l)}$ , then

$$\nabla_{\mathbf{W}^{(l)}} \mathcal{L} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$$

- Time complexity:  $O(|\theta|)$  — same order as a forward pass
- Made efficient by automatic differentiation (PyTorch, TensorFlow)



# Backpropagation

## Core Idea

Apply the **chain rule** repeatedly to compute  $\partial \mathcal{L} / \partial \mathbf{W}^{(l)}$  for every layer, from output back to input.

## Two-pass algorithm:

- 1 **Forward pass:** compute  $\mathbf{z}^{(l)}, \mathbf{a}^{(l)}$  for all  $l$ ; store activations
- 2 **Backward pass:** compute error signal  $\delta^{(l)}$ , then  

$$\nabla_{\mathbf{W}^{(l)}} \mathcal{L} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$$
  - Time complexity:  $O(|\theta|)$  — same order as a forward pass
  - Made efficient by automatic differentiation (PyTorch, TensorFlow)



# The Training Loop

- 1 **Initialise** weights  $\theta_0$  (random)
- 2 Split data: **train** / **validation** / **test**
- 3 For each **epoch**:
  - Sample a mini-batch  $\{(x_i, y_i)\}_{i=1}^B$
  - Forward pass  $\rightarrow$  compute  $\mathcal{L}$
  - Backward pass  $\rightarrow$  compute  $\nabla \mathcal{L}$
  - Update  $\theta \leftarrow \theta - \eta \nabla \mathcal{L}$
- 4 Monitor **validation loss**; stop when it increases (early stopping)

Golden Rule

Never tune hyperparameters using the **test set**. The test set measures final generalisation only.



# The Training Loop

- 1 **Initialise** weights  $\theta_0$  (random)
- 2 Split data: **train** / validation / test
- 3 For each **epoch**:
  - Sample a mini-batch  $\{(x_i, y_i)\}_{i=1}^B$
  - Forward pass  $\rightarrow$  compute  $\mathcal{L}$
  - Backward pass  $\rightarrow$  compute  $\nabla \mathcal{L}$
  - Update:  $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}$
- 4 Monitor **validation loss**; stop when it increases (early stopping)

## Golden Rule

Never tune hyperparameters using the **test set**. The test set measures final generalisation only.



# The Training Loop

- 1 **Initialise** weights  $\theta_0$  (random)
- 2 Split data: **train** / **validation** / **test**
- 3 For each **epoch**:

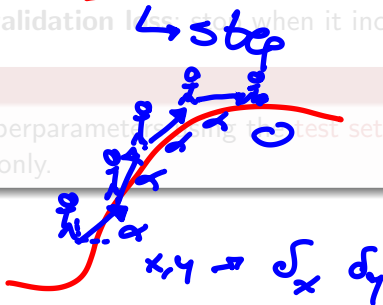
- Sample a mini-batch  $\{(x_i, y_i)\}_{i=1}^B$
- Forward pass  $\rightarrow$  compute  $\mathcal{L}$
- Backward pass  $\rightarrow$  compute  $\nabla \mathcal{L}$
- Update:  $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}$

*over all train data*

- 4 Monitor **validation loss**: stop when it increases (early stopping)

## Golden Rule

Never tune hyperparameters using the **test set**. The test set measures final generalisation only.



# The Training Loop

- 1 **Initialise** weights  $\theta_0$  (random)
- 2 Split data: **train** / **validation** / **test**
- 3 For each **epoch**:
  - Sample a mini-batch  $\{(x_i, y_i)\}_{i=1}^B$
  - Forward pass  $\rightarrow$  compute  $\mathcal{L}$
  - Backward pass  $\rightarrow$  compute  $\nabla \mathcal{L}$
  - Update:  $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}$
- 4 Monitor **validation loss**; stop when it increases (early stopping)

## Golden Rule

Never tune hyperparameters using the **test set**. The test set measures final generalisation only.



# The Training Loop

- ① **Initialise** weights  $\theta_0$  (random)
- ② Split data: **train** / **validation** / **test**
- ③ For each **epoch**:
  - Sample a mini-batch  $\{(x_i, y_i)\}_{i=1}^B$
  - Forward pass  $\rightarrow$  compute  $\mathcal{L}$
  - Backward pass  $\rightarrow$  compute  $\nabla \mathcal{L}$
  - Update:  $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}$
- ④ Monitor **validation loss**; stop when it increases (early stopping)

## Golden Rule

Never tune hyperparameters using the **test set**. The test set measures final generalisation only.

*overfitting*



# Practical Example: MLP on Two-Moons

- **Problem:** binary classification on non-linear boundary
- **Architecture:**  $2 \rightarrow 32 \rightarrow 32 \rightarrow 1$ , ReLU, sigmoid output
- **Experiment:** vary number of hidden layers and width
- **Observation:** deep narrow network generalises better than wide shallow one
- **Workshop connection:** understanding this experiment is the foundation for all classification tasks in Challenges 2, 5, and 7



# Practical Example: MLP on Two-Moons

- **Problem:** binary classification on non-linear boundary
- **Architecture:**  $2 \rightarrow 32 \rightarrow 32 \rightarrow 1$ , ReLU, sigmoid output
- **Experiment:** vary number of hidden layers and width
- **Observation:** deep narrow network generalises better than wide shallow one
- **Workshop connection:** understanding this experiment is the foundation for all classification tasks in Challenges 2, 5, and 7



# Practical Example: MLP on Two-Moons

- **Problem:** binary classification on non-linear boundary
- **Architecture:**  $2 \rightarrow 32 \rightarrow 32 \rightarrow 1$ , ReLU, sigmoid output
- **Experiment:** vary number of hidden layers and width
- **Observation:** deep narrow network generalises better than wide shallow one
- **Workshop connection:** understanding this experiment is the foundation for all classification tasks in Challenges 2, 5, and 7



# Practical Example: MLP on Two-Moons

- **Problem:** binary classification on non-linear boundary
- **Architecture:**  $2 \rightarrow 32 \rightarrow 32 \rightarrow 1$ , ReLU, sigmoid output
- **Experiment:** vary number of hidden layers and width
- **Observation:** deep narrow network generalises better than wide shallow one
- **Workshop connection:** understanding this experiment is the foundation for all classification tasks in Challenges 2, 5, and 7



# Practical Example: MLP on Two-Moons

- **Problem:** binary classification on non-linear boundary
- **Architecture:**  $2 \rightarrow 32 \rightarrow 32 \rightarrow 1$ , ReLU, sigmoid output
- **Experiment:** vary number of hidden layers and width
- **Observation:** deep narrow network generalises better than wide shallow one
- **Workshop connection:** understanding this experiment is the foundation for all classification tasks in Challenges 2, 5, and 7



# Outline

- 1 Neural Network Fundamentals
- 2 Optimization & Regularization



# Gradient Descent

## Update Rule

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}(\theta)$$

### Full-batch GD

$$\theta_0 = w_1, w_2, w_3, \dots, w_n$$

### Deterministic

$$\theta_1 = w_1, w_2, w_3, \dots, w_n$$

### Mini-batch SGD

- Noisy gradient estimates  $\rightarrow$  random
- Constant step size per step
- Noise helps escape local minima  $\rightarrow \nabla_{\theta}$
- Standard in practice

- Batch size  $B$ : typical values 32–512; affects convergence and memory
- Learning rate  $\alpha$  is the most critical hyperparameter

$$\nabla_{\theta} \approx \theta$$



# Gradient Descent

## Update Rule

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}(\theta)$$

### Full-batch GD

- Exact gradient
- Expensive per step
- Deterministic

### Mini-batch SGD

- Noisy gradient estimate
- Cheap per step
- Noise helps escape local minima
- Standard in practice



# Gradient Descent

## Update Rule

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}(\theta)$$

hyperparameter optimization

### Full-batch GD

- Exact gradient
- Expensive per step
- Deterministic

### Mini-batch SGD

- Noisy gradient estimate
- Cheap per step
- Noise helps escape local minima

$\bar{\nabla} \neq 0$

32-64-128-256-512

• Standard in practice

- Batch size  $B$ : typical values 32–512; affects convergence and memory
- Learning rate  $\alpha$  is the most critical hyperparameter

0.0003  
0.0001

0.003  
0.001

0.03  
0.01

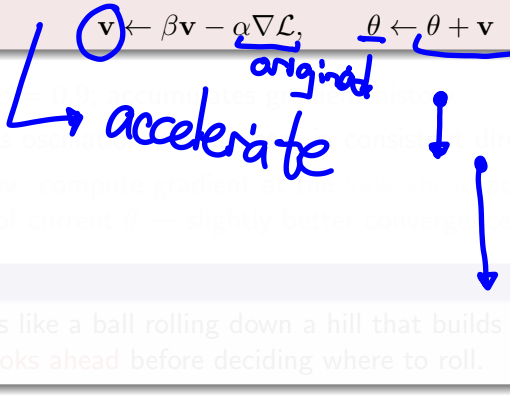
0.3  
0.1



# Momentum and Nesterov *~ 80's*

## SGD with Momentum

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla \mathcal{L}, \quad \theta \leftarrow \theta + \mathbf{v}$$



## Intuition

Momentum is like a ball rolling down a hill that builds *inertia*. Nesterov is a ball that *looks ahead* before deciding where to roll.



# Momentum and Nesterov

## SGD with Momentum

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla \mathcal{L}, \quad \theta \leftarrow \theta + \mathbf{v}$$

- Typical  $\beta = 0.9$ ; accumulates gradient **history**
- Dampens oscillations; accelerates in consistent directions
- **Nesterov**: compute gradient at the **look-ahead** position  $\theta + \beta \mathbf{v}$  instead of current  $\theta$  — slightly better convergence

## Intuition

Momentum is like a ball rolling down a hill that builds **inertia**. Nesterov is a ball that **looks ahead** before deciding where to roll.



# Momentum and Nesterov

## SGD with Momentum

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla \mathcal{L}, \quad \theta \leftarrow \theta + \mathbf{v}$$

- Typical  $\beta = 0.9$ ; accumulates gradient **history**
- Dampens oscillations; accelerates in consistent directions
- **Nesterov**: compute gradient at the **look ahead** position  $\theta + \beta \mathbf{v}$  instead of current  $\theta$  — slightly better convergence

### Intuition

Momentum is like a ball rolling down a hill that builds **inertia**. Nesterov is a ball that **looks ahead** before deciding where to roll.



# Momentum and Nesterov

## SGD with Momentum

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla \mathcal{L}, \quad \theta \leftarrow \theta + \mathbf{v}$$

- Typical  $\beta = 0.9$ ; accumulates gradient **history**
- Dampens oscillations; accelerates in consistent directions
- **Nesterov**: compute gradient at the **look-ahead** position  $\theta + \beta \mathbf{v}$  instead of current  $\theta$  — slightly better convergence

## Intuition

Momentum is like a ball rolling down a hill that builds **inertia**. Nesterov is a ball that **looks ahead** before deciding where to roll.



# Momentum and Nesterov

## SGD with Momentum

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla \mathcal{L}, \quad \theta \leftarrow \theta + \mathbf{v}$$

- Typical  $\beta = 0.9$ ; accumulates gradient **history**
- Dampens oscillations; accelerates in consistent directions
- **Nesterov**: compute gradient at the **look-ahead** position  $\theta + \beta \mathbf{v}$  instead of current  $\theta$  — slightly better convergence

## Intuition

Momentum is like a ball rolling down a hill that builds **inertia**. Nesterov is a ball that looks ahead before deciding where to roll.

↓  
snowball



# Adam Optimizer $\rightarrow$ XGBoost

Adaptive Moment Estimation (Kingma & Ba, 2015)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \underline{g_t}$$

$\swarrow$  *weighted avg.*

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta \leftarrow \theta - \underline{\alpha} \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

$$\hat{m}_t = m_t / (1 - \beta_1^t)$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$\swarrow$  *scaling*

$\swarrow$  *momentum*

$\swarrow$  *adaptive*

*Convex Combination*  
 $\alpha + (1 - \alpha)$



# Adam Optimizer

## Adaptive Moment Estimation (Kingma & Ba, 2015)

$$m_t = \underline{\beta_1} m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \underline{\beta_2} v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta \leftarrow \theta - \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

$$\hat{m}_t = m_t / (1 - \beta_1^t)$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

- Defaults:  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\varepsilon = 10^{-8}$ ,  $\alpha = 10^{-3}$   
0.001
- Per-parameter adaptive learning rates
- **AdamW**: Adam + decoupled weight decay — preferred in current practice
- Adam dominates in deep learning; SGD+Momentum preferred in CV research



# Adam Optimizer

## Adaptive Moment Estimation (Kingma & Ba, 2015)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\hat{m}_t = m_t / (1 - \beta_1^t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta \leftarrow \theta - \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

- Defaults:  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\varepsilon = 10^{-8}$ ,  $\alpha = 10^{-3}$
- Per-parameter **adaptive learning rates**
- **AdamW**: Adam + decoupled weight decay — preferred in current practice
- Adam dominates in deep learning; SGD+Momentum preferred in CV research



# Adam Optimizer

## Adaptive Moment Estimation (Kingma & Ba, 2015)

$$\begin{aligned}
 m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t & \hat{m}_t &= m_t / (1 - \beta_1^t) \\
 v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 & \hat{v}_t &= v_t / (1 - \beta_2^t) \\
 \theta &\leftarrow \theta - \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)
 \end{aligned}$$

- Defaults:  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\varepsilon = 10^{-8}$ ,  $\alpha = 10^{-3}$
- Per-parameter **adaptive learning rates**
- **AdamW**: Adam + decoupled weight decay — preferred in current practice
- Adam dominates in deep learning; SGD+Momentum preferred in CV research



# Adam Optimizer

## Adaptive Moment Estimation (Kingma & Ba, 2015)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\hat{m}_t = m_t / (1 - \beta_1^t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta \leftarrow \theta - \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

- Defaults:  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\varepsilon = 10^{-8}$ ,  $\alpha = 10^{-3}$
- Per-parameter **adaptive learning rates**
- **AdamW**: Adam + decoupled weight decay — preferred in current practice
- Adam dominates in deep learning; SGD + Momentum preferred in CV research



- ReduceLROnPlateau: halve LR when validation loss stops improving

Modern practice: **linear warmup + cosine decay**. Used in LLaMA, BERT, and GPT. Especially important with large batch sizes.

Converge fast  $\odot$

Optimal



# Learning Rate Scheduling

- **Step decay:** multiply LR by factor  $\gamma$  every  $k$  epochs
- **Cosine annealing**  $\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})(1 + \cos(\pi t/T))$
- **Linear warmup:** ramp from near-zero for first  $w$  steps, then decay
- **ReduceLROnPlateau:** halve LR when validation loss stops improving

## Warmup and Decay

Modern practice: linear warmup + cosine decay. Used in ResNets, BERT, and GPT. Especially important with large batch sizes.




# Learning Rate Scheduling

- **Step decay:** multiply LR by factor  $\gamma$  every  $k$  epochs
- **Cosine annealing:**  $\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})(1 + \cos(\pi t/T))$
- **Linear warmup:** ramp from near-zero for first  $w$  steps, then decay
- **ReduceLROnPlateau:** halve LR when validation loss stops improving

## Warmup and Decay

Modern practice: **linear warmup** + **cosine decay**. Used in ResNets, BERT, and GPT. Especially important with large batch sizes.



# Learning Rate Scheduling

- **Step decay:** multiply LR by factor  $\gamma$  every  $k$  epochs
- **Cosine annealing:**  $\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})(1 + \cos(\pi t/T))$
- **Linear warmup:** ramp from near-zero for first  $w$  steps, then decay
- **ReduceLROnPlateau:** halve LR when validation loss stops improving

## Warmup and Decay

Modern practice: linear warmup + cosine decay. Used in ResNets, BERT, and GPT. Especially important with large batch sizes.



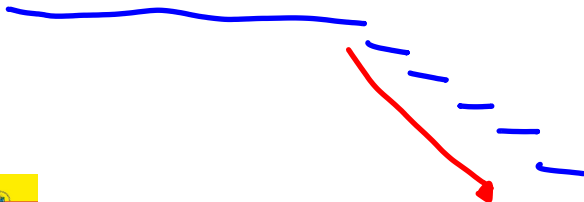
# Learning Rate Scheduling

- **Step decay:** multiply LR by factor  $\gamma$  every  $k$  epochs
- **Cosine annealing:**  $\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})(1 + \cos(\pi t/T))$
- **Linear warmup:** ramp from near-zero for first  $w$  steps, then decay
- **ReduceLROnPlateau:** halve LR when validation loss stops improving

## Warmup and Decay

Modern practice: linear warmup + cosine decay. Used in ResNets, BERT, and GPT. Especially important with large batch sizes.

Google



# Bias-Variance Trade-Off

## Underfitting (high bias)

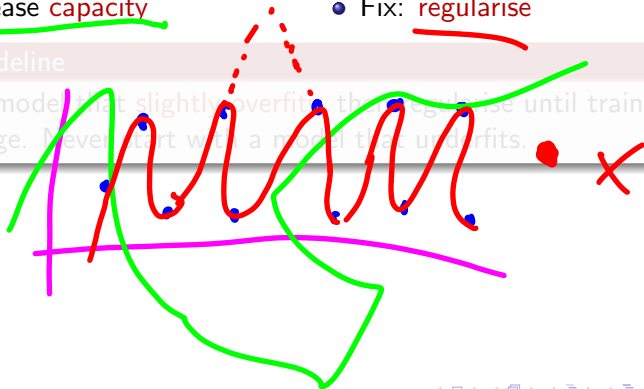
- Model too simple
- High training error
- Cannot capture data structure
- Fix: increase capacity

## Overfitting (high variance)

- Model too complex
- Low training, high val error
- Memorises noise
- Fix: regularise

### Practical Guideline

Start with a model that slightly overfits, then regularise until train and val errors converge. Never start with a model that underfits.



# Bias-Variance Trade-Off

## Underfitting (high bias)

- Model too simple
- High training error
- Cannot capture data structure
- Fix: increase **capacity**

## Overfitting (high variance)

- Model too complex
- Low training, high val error
- Memorises noise
- Fix: **regularise**

### Practical Guideline

Start with a model that **slightly overfits**, then regularise until train and val errors converge. Never start with a model that underfits.

$\alpha$ -medium  $\Leftarrow$  Control



## L1 and L2 Regularization

→ training time

## Augmented Objective

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \Omega(\theta)$$

 $\mathcal{L} \rightarrow \Omega(\theta)$ 

## L2 Regularization (Weight Decay / Ridge)

$$\Omega(\theta) = \frac{\lambda}{2} \|\mathbf{W}\|_F^2 \quad \text{drives weights small but rarely zero}$$

 $\mathcal{L} \rightarrow -\Omega(\theta)$ 

## L1 Regularization (Lasso)

$$\Omega(\theta) = \lambda \|\mathbf{W}\|_1 \quad \text{promotes sparse weights (many exact zeros)}$$



# L1 and L2 Regularization

## Augmented Objective

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \Omega(\theta)$$

## L2 Regularization (Weight Decay / Ridge)

$$\Omega(\theta) = \frac{\lambda}{2} \|\mathbf{W}\|_F^2 \quad \text{drives weights small but rarely zero}$$

## L1 Regularization (Lasso)

$$\Omega(\theta) = \lambda \|\mathbf{W}\|_1 \quad \text{promotes sparse weights (many exact zeros)}$$

- In deep learning: L2 (weight decay) is the standard choice
- Elastic Net: combines both — used in linear models and sparse NNs



# L1 and L2 Regularization

## Augmented Objective

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \Omega(\theta)$$

## L2 Regularization (Weight Decay / Ridge)

$$\Omega(\theta) = \frac{\lambda}{2} \|\mathbf{W}\|_F^2 \quad \text{drives weights small but rarely zero}$$

## L1 Regularization (Lasso)

$$\Omega(\theta) = \lambda \|\mathbf{W}\|_1 \quad \text{promotes sparse weights (many exact zeros)}$$

edge to size



# L1 and L2 Regularization

## Augmented Objective

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \Omega(\theta)$$

## L2 Regularization (Weight Decay / Ridge)

$$\Omega(\theta) = \frac{\lambda}{2} \|\mathbf{W}\|_F^2 \quad \text{drives weights small but rarely zero}$$

## L1 Regularization (Lasso)

$$\Omega(\theta) = \lambda \|\mathbf{W}\|_1 \quad \text{promotes sparse weights (many exact zeros)}$$

- In deep learning: L2 (weight decay) is the standard choice
- Elastic Net: combines both — used in linear models and sparse NNs



# Dropout

## Definition (Srivastava et al., 2014)

During training, independently zero out each activation with probability  $p$ . At inference, multiply activations by  $(1 - p)$  (*inverted dropout* scales during training).

- Forces redundant representations — no single neuron can be relied upon
- Equivalent to averaging over  $2^n$  thinned networks
- Typical  $p = 0.5$  (hidden layers);  $p = 0.1$  after conv layers
- Less effective with  $p > 0.5$  (neurons on the same layer)



# Dropout

## Definition (Srivastava et al., 2014)

During training, independently zero out each activation with probability  $p$ . At inference, multiply activations by  $(1 - p)$  (*inverted dropout* scales during training).

- Forces **redundant representations** — no single neuron can be relied upon
- Equivalent to averaging over  $2^n$  thinned networks
- Typical  $p = 0.5$  (hidden layers);  $p = 0.1$  after conv layers
- Less effective with **Batch Normalization** on the same layer



# Dropout

## Definition (Srivastava et al., 2014)

During training, independently zero out each activation with probability  $p$ . At inference, multiply activations by  $(1 - p)$  (*inverted dropout* scales during training).

- Forces **redundant representations** — no single neuron can be relied upon
- Equivalent to averaging over  $2^n$  thinned networks
  - Typical  $p = 0.5$  (hidden layers);  $p = 0.1$  after conv layers
  - Less effective with **Batch Normalization** on the same layer



# Dropout

## Definition (Srivastava et al., 2014)

During training, independently zero out each activation with probability  $p$ . At inference, multiply activations by  $(1 - p)$  (*inverted dropout* scales during training).

- Forces **redundant representations** — no single neuron can be relied upon
- Equivalent to averaging over  $2^n$  thinned networks
- Typical  $p = 0.5$  (hidden layers);  $p = 0.1$  after conv layers
- Less effective with **Batch Normalization** on the same layer



# Dropout

## Definition (Srivastava et al., 2014)

During training, independently zero out each activation with probability  $p$ . At inference, multiply activations by  $(1 - p)$  (*inverted dropout* scales during training).

- Forces **redundant representations** — no single neuron can be relied upon
- Equivalent to averaging over  $2^n$  thinned networks
- Typical  $p = 0.5$  (hidden layers);  $p = 0.1$  after conv layers
- Less effective with **Batch Normalization** on the same layer



# Batch Normalization

## Definition (Ioffe & Szegedy, 2015)

Normalise activations within a mini-batch, then **scale and shift**:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- $\gamma, \beta$  are learnable parameters
- Stabilises training: allows larger learning rates
- Reduces sensitivity to weight initialisation
- At inference: use running mean/variance accumulated during training
- Alternative: Layer Norm (Transformers), Group Norm (small batches)



# Batch Normalization

Definition (Ioffe & Szegedy, 2015)

Normalise activations within a mini-batch, then **scale and shift**:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- $\gamma, \beta$  are **learnable** parameters
- Stabilises training: allows larger learning rates
- Reduces sensitivity to weight initialisation
- At inference: use running mean/variance accumulated during training
- Alternative: Layer Norm (Transformers), Group Norm (small batches)



# Batch Normalization

## Definition (Ioffe & Szegedy, 2015)

Normalise activations within a mini-batch, then **scale and shift**:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- $\gamma, \beta$  are **learnable** parameters
- Stabilises training: allows larger learning rates
- Reduces sensitivity to weight initialisation
- At inference: use running mean/variance accumulated during training
- Alternative: Layer Norm (Transformers), Group Norm (small batches)



# Batch Normalization

## Definition (Ioffe & Szegedy, 2015)

Normalise activations within a mini-batch, then **scale and shift**:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- $\gamma, \beta$  are **learnable** parameters
- Stabilises training: allows larger learning rates
- Reduces sensitivity to weight initialisation
- At inference: use running mean/variance accumulated during training
- Alternative: Layer Norm (Transformers), Group Norm (small batches)



# Batch Normalization

## Definition (Ioffe & Szegedy, 2015)

Normalise activations within a mini-batch, then **scale and shift**:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- $\gamma, \beta$  are **learnable** parameters
- Stabilises training: allows larger learning rates
- Reduces sensitivity to weight initialisation
- At inference: use running mean/variance accumulated during training
- Alternative: Layer Norm (Transformers), Group Norm (small batches)



# Batch Normalization

## Definition (Ioffe & Szegedy, 2015)

Normalise activations within a mini-batch, then **scale and shift**:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

- $\gamma, \beta$  are **learnable** parameters
- Stabilises training: allows larger learning rates
- Reduces sensitivity to weight initialisation
- At inference: use running mean/variance accumulated during training
- Alternative: Layer Norm (Transformers), Group Norm (small batches)



# Practical Example: Regularizing a Deep MLP

- **Problem:** overfit a 5-layer MLP on a small dataset
- **Experiment A:** add L2 weight decay  $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
- **Experiment B:** add Dropout  $p \in \{0.2, 0.4, 0.6\}$
- **Experiment C:** insert Batch Normalization before each activation
- **Metric:** validation accuracy and gap to training accuracy
- **Workshop connection:** regularisation setup is required in Challenges 3, 4, 5, and 6



# Practical Example: Regularizing a Deep MLP

- **Problem:** overfit a 5-layer MLP on a small dataset
- **Experiment A:** add L2 weight decay  $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
- **Experiment B:** add Dropout  $p \in \{0.2, 0.4, 0.6\}$
- **Experiment C:** insert Batch Normalization before each activation
- **Metric:** validation accuracy and gap to training accuracy
- **Workshop connection:** regularisation setup is required in Challenges 3, 4, 5, and 6



# Practical Example: Regularizing a Deep MLP

- **Problem:** overfit a 5-layer MLP on a small dataset
- **Experiment A:** add L2 weight decay  $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
- **Experiment B:** add Dropout  $p \in \{0.2, 0.4, 0.6\}$
- **Experiment C:** insert Batch Normalization before each activation
- **Metric:** validation accuracy and gap to training accuracy
- **Workshop connection:** regularisation setup is required in Challenges 3, 4, 5, and 6



# Practical Example: Regularizing a Deep MLP

- **Problem:** overfit a 5-layer MLP on a small dataset
- **Experiment A:** add L2 weight decay  $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
- **Experiment B:** add Dropout  $p \in \{0.2, 0.4, 0.6\}$
- **Experiment C:** insert Batch Normalization before each activation
- **Metric:** validation accuracy and gap to training accuracy
- **Workshop connection:** regularisation setup is required in Challenges 3, 4, 5, and 6



# Practical Example: Regularizing a Deep MLP

- **Problem:** overfit a 5-layer MLP on a small dataset
- **Experiment A:** add L2 weight decay  $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
- **Experiment B:** add Dropout  $p \in \{0.2, 0.4, 0.6\}$
- **Experiment C:** insert Batch Normalization before each activation
- **Metric:** validation accuracy and gap to training accuracy
- **Workshop connection:** regularisation setup is required in Challenges 3, 4, 5, and 6



# Outline

- 1 Neural Network Fundamentals
- 2 Optimization & Regularization



# Thanks!

## Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/machine-learning>

